

第二章：模型评估 与选择

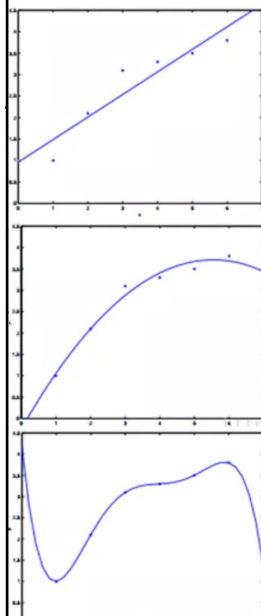
1

大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

2

误差



3

误差

- 机器学习的主要挑战在于在未见过的数据输入上表现良好，这个能力称为泛化能力 (generalization) 。
- 而我们的机器学习模型都是从训练集中学习参数得到的，如何确保其在和训练集“隔离”的测试集中表现良好呢？

4

经验误差与过拟合

□ 错误率&误差：

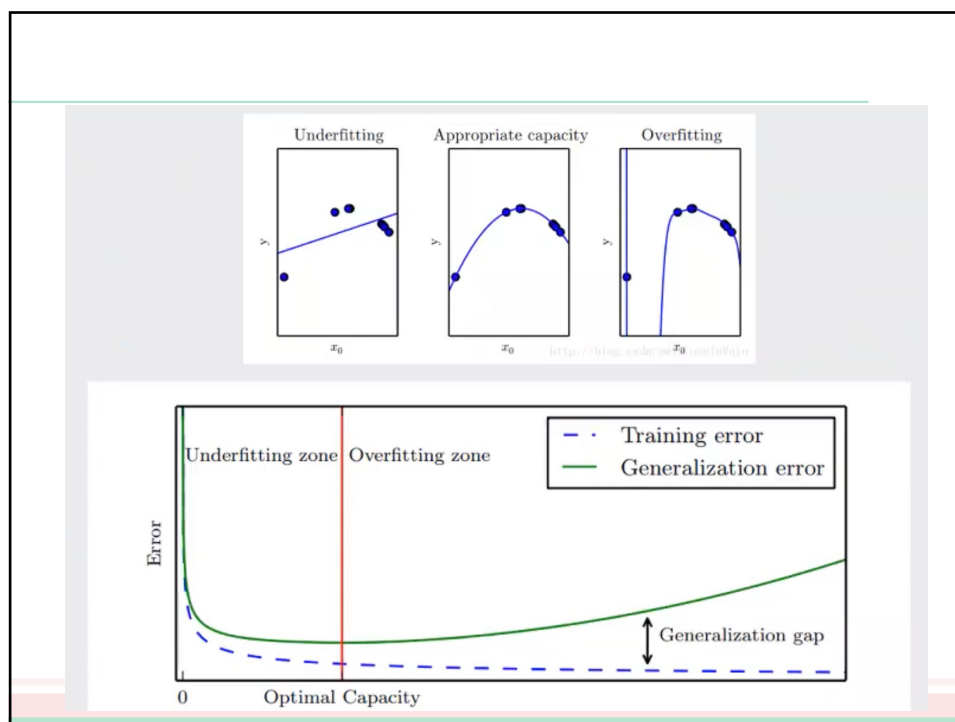
- 错误率：错分样本的占比 $E = a/m$
- 误差：样本真实输出与预测输出之间的差异
 - 训练(经验)误差：训练集上
 - 测试误差：测试集
 - 泛化误差：除训练集外所有样本

由于事先并不知道新样本的特征，我们只能努力使经验误差最小化；

很多时候虽然能在训练集上做到分类错误率为零，但多数情况下这样的学习器并不好

我们希望泛化误差小的学习器

5



6

经验误差与过拟合

□ 过拟合:

学习器把训练样本学习的“太好”，将训练样本本身的特点 当做所有样本的一般性质，导致泛化性能下降

- 优化目标加正则项
- early stop

□ 欠拟合:

对训练样本的普遍规律尚未学好

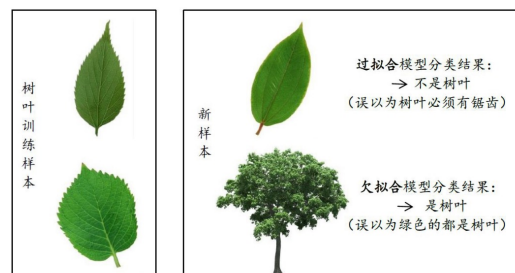
- 决策树:拓展分支
- 神经网络:增加训练轮数

欠拟合可以通过增加模型的复杂度来解决。

过拟合是机器学习的关键障碍且不可避免！！

7

经验误差与过拟合



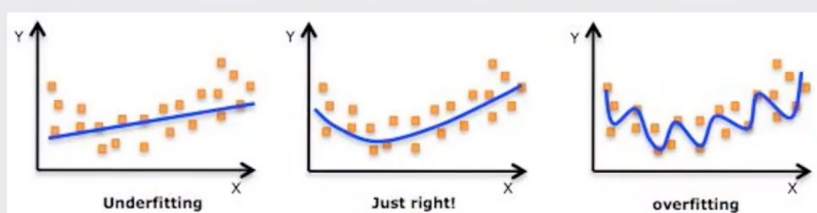
过拟合、欠拟合的直观类比

过拟合：学习器把训练样本本身特点当做所有潜在样本都会具有的一般性质。

欠拟合：训练样本的一般性质尚未被学习器学好。

8

- 上学考试的时候，有的人采取题海战术，把每个题目都背下来。但是题目稍微一变，他就不会做了。
- 因为他非常复杂的记住了每道题的做法，而没有抽象出通用的规则。



9

大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

10

学习器泛化评估方法

现实任务中往往会**对学习器的泛化性能**、时间开销、存储开销、可解释性等方面的因素**进行评估并做出选择**

训练集S和测试集T组成数据集D.

我们假设**测试集**是从样本真实分布中独立采样获得(保证测试集和训练集数据**分布一样**)，将测试集上的“**测试误差**”作为**泛化误差的近似**

测试集要和训练集中的样本尽量**互斥**。

11

常用评估方法

测试方法	数学表达	注意事项	优缺点
留出法 (hold-out)	$D = S \cup T$ $S \cap T = \emptyset$	分层采样 (stratified sampling) 重复试验取平均评估结果	测试集小, 评估结果方差较大 训练集小, 评估结果偏差较大
交叉验证法 (cross validation)	$D = D_1 \cup \dots \cup D_k$ $D_i \cap D_j = \emptyset (i \neq j)$	p 次 k 折交叉验证	稳定性和保真性很大程度取决于 k
留一法 (Leave-One-Out, LOO)	$D = D_1 \cup \dots \cup D_k$ $D_i \cap D_j = \emptyset (i \neq j)$ $k = D $	每次使用一个样本验证	不受随机样本划分方式影响 数据量大时计算量大
自助法 (bootstrapping)	$ S = D $ $T = D \setminus S$	可重复采样/有放回采样	数据集较小有用 改变初始数据集的分布, 引入偏差

12

评估方法

通常将包含个 m 样本的数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ 拆分成训练集 S 和测试集 T :

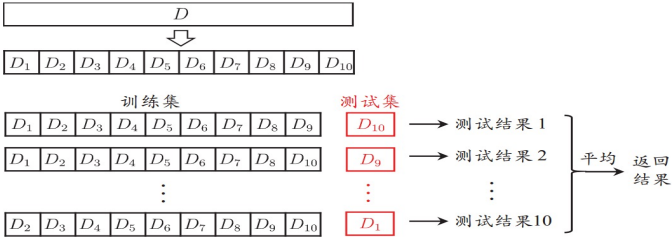
- 留出法(Hold-out) :
- 直接将数据集划分为两个互斥集合
 - 训练/测试集划分要尽可能保持数据分布的一致性
 - 一般若干次随机划分、重复实验取平均值
 - 训练/测试样本比例通常为2:1~4:1

测试集小时, 评估结果方差较大
训练集小时, 评估结果偏差较大

13

评估方法

- 交叉验证法(cross validation) :
- 将数据集分层采样划分为 k 个大小相似的互斥子集, 每次用 $k-1$ 个子集的并集作为训练集, 余下的子集作为测试集, 最终返回 k 个测试结果的均值, k 最常用的取值是10.



10 折交叉验证示意图

与留出法类似, 将数据集 D 划分为 k 个子集同样存在多种划分方式, 为了减小因样本划分不同而引入的差别, k 折交叉验证通常随机使用不同的划分重复 p 次, 最终的评估结果是这 p 次 k 折交叉验证结果的均值, 例如常见的 “10 次10折交叉验证”

14

留一法(leave-one-out)

假设数据集 D 包含 m 个样本，若令 $k = m$ ，即数据集划分为 m 个互斥子集，**每次使用一个样本进行验证**，则得到留一法：

- 不受随机样本划分方式的影响
- 结果往往比较准确
- 当数据集比较大时，计算开销难以忍受

15

评估方法

□ 自助法 (bootstrapping) :

以自助采样法为基础，对数据集 D 有放回采样 m 次得到训练集 D' ， $D \setminus D'$ 用做测试集。

- 实际模型与预期模型都使用 m 个训练样本
- 约有1/3的样本没在训练集中出现
- 从初始数据集中产生多个不同的训练集，对集成学习有很大的好处
- 自助法在**数据集较小**、难以有效划分训练/测试集时很有用；由于改变了数据集分布可能引入估计偏差，在数据量足够时，留出法和交叉验证法更常用。

例如医疗、反恐防暴

16

常用评估方法

测试方法	数学表达	注意事项	优缺点
留出法 (hold-out)	$D = S \cup T$ $S \cap T = \emptyset$	分层采样 (stratified sampling) 重复试验取平均评估结果	测试集小, 评估结果方差较大 训练集小, 评估结果偏差较大
交叉验证法 (cross validation)	$D = D_1 \cup \dots \cup D_k$ $D_i \cap D_j = \emptyset (i \neq j)$	p 次 k 折交叉验证	稳定性和保真性很大程度取决于 k
留一法 (Leave-One-Out, LOO)	$D = D_1 \cup \dots \cup D_k$ $D_i \cap D_j = \emptyset (i \neq j)$ $k = D $	每次使用一个样本验证	不受随机样本划分方式影响 数据量大时计算量大
自助法 (bootstrapping)	$ S = D $ $T = D \setminus S$	可重复采样/有放回采样	数据集较小有用 改变初始数据集的分布, 引入偏差

17

调参与最终模型

- 参数不同，模型的性能也会有显著差异
- 参数调节 (parameter tuning)
- 算法参数 (超参数)：人工设定候选值，它们无法从数据中直接学习，需要通过实验或优化方法调整 (如正则化参数值，迭代次数，学习率)
 - 模型参数：通过数据自动学习 (如线性回归中的权重，神经元连接权重)
- 学习算法和参数配置确定后要用整个数据集重新训练模型
- 数据集
- 训练集 (training set)：训练估计模型
 - 验证集 (validation set)：模型参数调整
 - 测试集 (testing set)：估计泛化能力

18

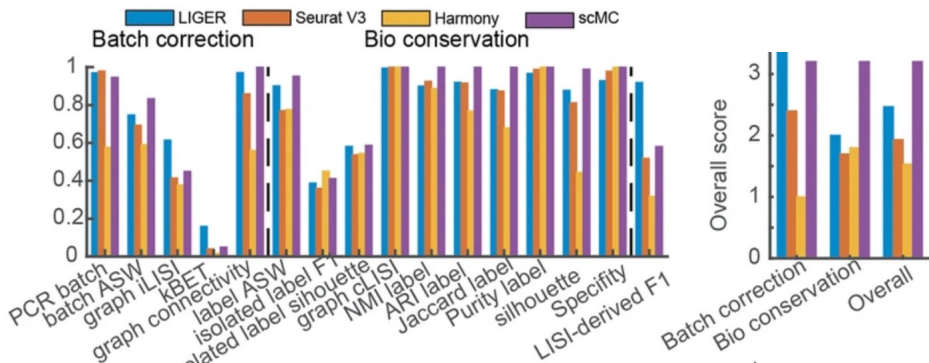
大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

19

性能度量

性能度量是衡量模型泛化能力的评价标准，反映了任务需求；
使用不同的性能度量往往会导致不同的评判结果



Zhang and Nie. Genome Biology 2021, 22(1):10.

20

性能度量

对于**分类任务**,错误率和精度是最常用的两种性能度量：

- 错误率 (Error rate)：分错样本占样本总数的比例
- 精度 (Accuracy)：分对样本占样本总数的比例

分类错误率

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) \neq y_i)$$

精度

$$\begin{aligned} \text{acc}(f; D) &= \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) = y_i) \\ &= 1 - E(f; D) . \end{aligned}$$

错误率和精度是否可以回答：

- 1) 挑出来的西瓜中有多少比例是好瓜
- 2) 所有好瓜中有多少比例被挑出来了

21

TP、FN、FP、TN

我们以西瓜好坏的预测为例，正例：好瓜，反例：坏瓜，对于二分类问题，将样本依据真实的类别和分类器的预测分别进行组合，可分为四种情况：

TP、FN、FP、TN的含义：

TP (true positive)：将正例预测为正例的样本数；

FN (false negative)：将正例预测为反例的样本数；

FP (false positive)：将反例预测为正例的样本数；

TN (true negative)：将反例预测为反例的样本数；

true、false表示预测结果的正确与错误，positive、negative表示预测样本的正例与反例。

“混淆矩阵” (confusion matrix)
分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

错误率 (error rate)：

$$\text{errorrate} = \frac{FN+FP}{TP+FN+FP+TN}$$

准确率 (accuracy)

$$\text{accuracy} = \frac{TP+TN}{TP+FN+FP+TN}$$

22

性能度量

错误率和精度无法回答：
1) 挑出来的西瓜中有多少比例是好瓜
2) 所有好瓜中有多少比例被挑出来了

查准率(precision，准确率)：预测出来的正例中正确的比例
查全率(recall，召回率)：正例被预测出来的比率

统计真实标记和预测结果的组合可以得到“混淆矩阵”
(confusion matrix)

分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

查准率 (precision) $P = \frac{TP}{TP + FP}$

查全率 (recall) $R = \frac{TP}{TP + FN}$

23

性能度量

F1度量（基于查准率和查全率的调和平均值）：

$$\frac{1}{F_1} = \frac{1}{2} \cdot \left(\frac{1}{P} + \frac{1}{R} \right) \quad \Rightarrow \quad F_1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times TP}{\text{样例总数} + TP - TN}$$

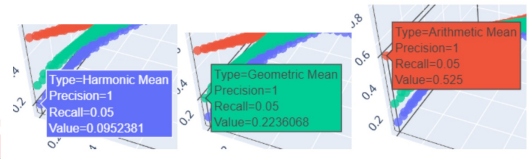
比F1更一般的形式 F_β ,

$$\frac{1}{F_\beta} = \frac{1}{1 + \beta^2} \cdot \left(\frac{1}{P} + \frac{\beta^2}{R} \right) \quad F_\beta = \frac{(1 + \beta^2) \times P \times R}{(\beta^2 \times P) + R}$$

- $\beta = 1$: 标准F1
- $\beta > 1$: 偏重查全率(逃犯信息检索，尽可能减少漏掉逃犯)
- $\beta < 1$: 偏重查准率(商品推荐系统，推荐内容确实是用户感兴趣的)

为什么用调和平均，而不是算术平均或几何平均？

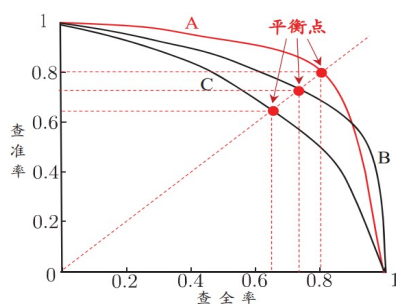
调和平均更重视较小值



24

性能度量-Precision-Recall曲线

根据学习器的预测结果按正例可能性大小对样例进行排序（排在最前面的是认为“最可能”是正例的样本），并按此顺序逐个把样本作为正例进行预测，则每次可以计算出当前的查准率和查全率，于是可以得到查准率-查全率曲线简称“P-R曲线”



P-R曲线与平衡点示意图

如何比较两个曲线？

- 1) 一个曲线被另外一个完全“包住”
- 2) 比较P-R曲线下的面积（不易计算）

设计综合考虑查准率和查全率的性能度量：

- 1) 平衡点是曲线上“查准率=查全率”时的取值，可用来度量P-R曲线有交叉的分类器性能高低
- 2) F1 score

25

性能度量-ROC曲线

类似P-R曲线，根据学习器的预测结果对样例排序，并逐个作为正例进行预测，以“假正例率”为横轴，“真正例率”为纵轴可得到Receiver Operator Characteristic (ROC) 曲线, 全称“受试者工作特征”。

- 横轴——假正例率： $FPR = \frac{FP}{TN+FP}$
- 纵轴——真正例率： $TPR = \frac{TP}{TP+FN}$

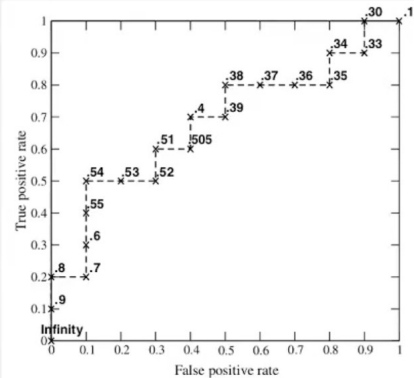
(1) 真正例率(True Postive Rate)TPR: $TP/(TP+FN)$, 代表分类器预测的正类中实际正实例占有所有正实例的比例。Sensitivity

(2) 假正例率(False Postive Rate)FPR: $FP/(FP+TN)$, 代表分类器预测的正类中实际负实例占有所有负实例的比例。1-Specificity

26

ROC图的绘制

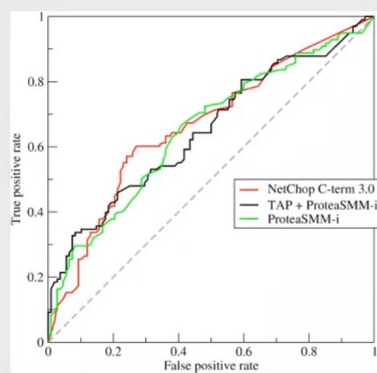
Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1



图中有20个测试样本，“class”表示每个样本真正的标签（p是正样本，n是负样本），Score是每个样本属于正样本的概率。接下来，从高到低，依次将“score”值作为阈值threshold。每次选取不同的threshold，就可以得到一组FPR, TPR。

27

ROC曲线



考虑ROC曲线图中的四个点和一条线。

第一个点， $(0, 1)$ ，即FPR=0, TPR=1，这意味着FN (false negative) =0，并且FP (false positive) =0。这是一个完美的分类器，它将所有的样本都正确分类。

第二个点， $(1, 0)$ ，即FPR=1, TPR=0，类似地分析可以发现这是一个最糟糕的分类器，因为它成功避开了所有的正确答案。

第三个点， $(0, 0)$ ，即FPR=TPR=0，即FP (false positive) =TP (true positive) =0，可以发现该分类器预测所有的样本都为负样本 (negative)。

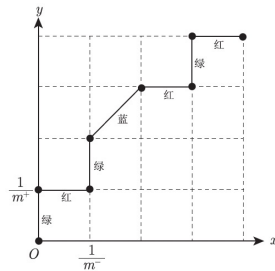
第四个点 $(1, 1)$ ，分类器实际上预测所有的样本都为正样本。

经过以上的分析，我们可以断言，ROC曲线越接近左上角，该分类器的性能越好。

28

性能度量

若某个学习器的ROC曲线被另一个学习器的曲线“包住”，则后者性能优于前者；否则如果曲线交叉，可以根据ROC曲线下面积大小进行比较，也即AUC值。



假设ROC曲线由 $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ 的点按序连接而形成 ($x_1 = 0, x_m = 1$)
AUC可用梯形公式估算为：

$$AUC = \frac{1}{2} \sum_{i=1}^{m-1} (x_{i+1} - x_i) \cdot (y_i + y_{i+1})$$

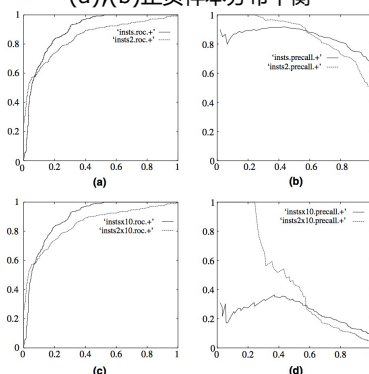
AUC衡量了样本预测的排序质量。

29

ROC曲线 vs Precision-Recall曲线

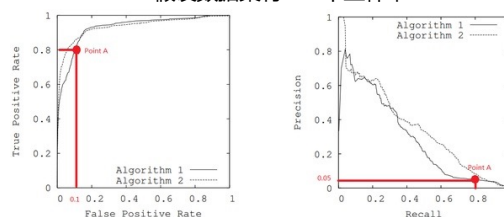
- 对于正负样本分布大致均匀的问题，ROC曲线作为性能指标更鲁棒
- 在正负样本分布得极不均匀，负例远大于正例时，PRC比ROC能更有效地反应分类器的好坏

(a),(b)正负样本分布平衡



(c),(d) 将负样本的数量增加到原来的10倍

假设数据集有100个正样本



(a) Comparison in ROC space

(b) Comparison in PR space

TPR=TP/(TP+FN)=TP/所有正样本

查准率 (precision) : TP/(TP+FP)

负正例率FPR: FP/(FP+TN)=FP/所有负样本

FPR因为负例基数大的缘故依然很小

30

多分类ROC曲线

在多分类任务中利用one vs rest方式计算各个类别的混淆矩阵，使用如下平均方式

- Macro（宏）：分别求出每一个二分类的指标，再进行算术平均
优点：直观、易懂，并且方便实现
缺点：实际情况下可能不同类别拥有不同的重要性，求平均会导致计算结果受不常用类别的影响
- Micro（微）：全局计算。将所有混淆矩阵累加在一起求平均，然后计算TPR/FPR/AUC

参考：https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html#sklearn.metrics.roc_auc_score

31

代价敏感错误率

现实任务中不同类型的错误所造成的后果很可能不同，如错误地把患者诊断为健康人 vs. 错误地把健康人诊断为患者

为了权衡不同类型错误所造成的不同损失，可为错误赋予“非均等代价”。

- 代价敏感错误率：加权的错误率

表 2.2 二分类代价矩阵

真实类别	预测类别	
	第 0 类	第 1 类
第 0 类	0	$cost_{01}$
第 1 类	$cost_{10}$	0

在非均等代价下，不再最小化错误次数，而是最小化“总体代价”，则“代价敏感”错误率相应的为：

$$E(f; D; cost) = \frac{1}{m} \left(\sum_{\mathbf{x}_i \in D^+} \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \times cost_{01} + \sum_{\mathbf{x}_i \in D^-} \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \times cost_{10} \right)$$

32

代价曲线

在非均等代价下，ROC曲线不能直接反映出学习器的期望总体代价，而“代价曲线”可以。

代价曲线的横轴是取值为[0,1]的正例概率代价

$$P(+)\text{cost} = \frac{p \times \text{cost}_{01}}{p \times \text{cost}_{01} + (1-p) \times \text{cost}_{10}}$$

纵轴是取值为[0,1]的归一化代价

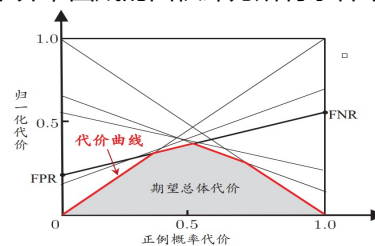
$$\text{cost}_{\text{norm}} = \frac{\text{FNR} \times p \times \text{cost}_{01} + \text{FPR} \times (1-p) \times \text{cost}_{10}}{p \times \text{cost}_{01} + (1-p) \times \text{cost}_{10}}$$

p 是样例为正例的概率

33

代价曲线

代价曲线图的绘制：ROC曲线上每个点对应了代价曲线上的一条线段，设ROC曲线上点的坐标为(FPR, TPR),则可相应计算出FNR,然后在代价平面上绘制一条从(0, FPR)到(1, FNR)的线段，线段下的面积即表示了该条件下的期望总体代价；如此将ROC曲线上的每个点转化为代价平面上的一条线段，然后取所有线段的下界，围成的面积即为所有条件下学习器的期望总体代价。



代价曲线与期望总体代价

ROC曲线上的一个点固定了模型的判别能力（TPR和FPR），而代价曲线上的一个点则需要同时考虑模型的判别能力和应用场景的代价与先验概率。当应用场景（代价和先验概率）变化时，同一个模型能力（ROC点）会计算出不同的归一化期望代价，这些代价值连成一条线段。

34

大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

35

性能评估

- 关于性能比较：
 - 测试性能并不等于泛化性能
 - 测试性能随着测试集的变化而变化
 - 很多机器学习算法本身有一定的随机性

有没有适当的方法对学习器的性能进行比较？ 直接比较 vs. 统计假设检验

假设检验为学习器性能比较提供了重要依据，基于其结果我们可以推断出若在测试集上观察到学习器A比B好，则A的泛化性能是否在统计意义上优于B，以及这个结论的把握有多大。

36

单个学习器泛化能力的t检验

面对多次重复留出法或者交叉验证法进行多次训练/测试时可使用“t检验”。

假定得到了k个测试错误率 $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_k$ ，则根据平均测试错误率 μ 和方差 σ ，假设泛化误差率为 ϵ_0 ，构造t统计量

$$\tau_t = \frac{\mu - \epsilon_0}{\sigma / \sqrt{k}}$$

则 τ_t 服从自由度为k-1的t分布

假定得到了k个测试错误率 $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_k$ ，假设 $\epsilon = \epsilon_0$ 对于显著度 α ，若 τ_t 位于临界范围 $[t_{-\alpha/2}, t_{\alpha/2}]$ 内，则假设不能被拒绝，即可认为泛化错误率 $\epsilon = \epsilon_0$ ，其置信度为 $1 - \alpha$ 。

38

不同学习器的交叉验证t检验

现实任务中，更多时候需要对不同学习器的性能进行比较

对两个学习器A和B，若k折交叉验证得到的测试错误率分别为 $\epsilon_1^A, \dots, \epsilon_k^A$ 和 $\epsilon_1^B, \dots, \epsilon_k^B$ ，可用k折交叉验证“成对t检验”进行比较检验。若两个学习器的性能相同，则他们使用相同的训练/测试集得到的测试错误率应相同，即 $\epsilon_i^A = \epsilon_i^B$

39

不同学习器的交叉验证t检验

先对每对结果求差 $\Delta_i = \epsilon_i^A - \epsilon_i^B$, 若两个学习器性能相同, 则差值应该为0, 继而用 $\Delta_1, \dots, \Delta_k$ 来对 “学习器A与B性能相同” 这个假设做t检验。计算出差值的均值 μ 和方差 σ

$$\tau_t = \left| \frac{\mu}{\sigma/\sqrt{k}} \right|$$

若 τ_t 小于临界值 $\tau_{\frac{\alpha}{2}, k-1}$, 则假设不能拒绝, 即认为两个学习器的性能没有显著差异

假设检验的前提是测试错误率为泛化错误率的独立采样, 然而由于样本有限, 使用交叉验证导致训练集重叠, **测试错误率并不独立, 从而过高估计假设成立的概率**, 为缓解这一问题, 可采用 “5*2交叉验证” 法 (5次二折)。

40

不同学习器的5*2交叉验证法

所谓5*2折交叉验证就是做5次二折交叉验证, 每次二折交叉验证之前将数据打乱, 使得5次交叉验证中的数据划分不重复。为缓解测试数据错误率的非独立性, 仅计算第一次2折交叉验证结果的平均值 $\mu = 0.5(\Delta_1^1 + \Delta_1^2)$ 和每次二折实验计算得到的方差 $\sigma_i^2 = \left(\Delta_i^1 - \frac{\Delta_i^1 + \Delta_i^2}{2}\right)^2 + \left(\Delta_i^2 - \frac{\Delta_i^1 + \Delta_i^2}{2}\right)^2$

$$\tau_t = \frac{\mu}{\sqrt{0.2 \sum_{i=1}^5 \sigma_i^2}}$$

服从自由度为5的t分布。

41

McNemar检验（麦克尼马尔/配对卡方检验）

对于二分类问题，**留出法**不仅可以估计出学习器A和B的测试错误率，还能获得两学习器分类结果的差别，如下表所示

两学习器分类差别列联表

算法 B	算法 A	
	正确	错误
正确	e_{00}	e_{01}
错误	e_{10}	e_{11}

两者都正确、都错误的样本数

假设两学习器性能相同 $e_{01} = e_{10}$

则 $|e_{01} - e_{10}|$ 应服从正态分布，McNemar检验（配对卡方检验）考虑变量

$$\tau_{\chi^2} = \frac{(|e_{01} - e_{10}| - 1)^2}{e_{01} + e_{10}}$$

服从自由度为1的 χ^2 分布。

42

Friedman（弗里德曼）检验

交叉验证t检验和McNemar检验都是在一个数据集上比较两个算法的性能，可以用Friedman检验在一组数据集上对多个算法进行比较。

假定用 D_1, D_2, D_3, D_4 四个数据集对算法 A, B, C 进行比较。

- 先使用留出法或者交叉验证法得到每个算法在每个数据集上的测试结果，
- 然后在每个数据集上根据性能由好到坏排序，并赋序值1,2,...;
- 若算法性能相同则平分序值，继而得到每个算法的平均序值 r_i

43

Friedman检验

得到表格如下所示，由平均序值进行Friedman检验来判断这些算法是否性能都相同。

算法比较序值表			
数据集	算法 A	算法 B	算法 C
D_1	1	2	3
D_2	1	2.5	2.5
D_3	1	2	3
D_4	1	2	3
平均序值	1	2.125	2.875

D1: A, B, C
D2: A, B/C

则变量：

$$\tau_{\chi^2} = \frac{12N}{k(k+1)} \left(\sum_{i=1}^k r_i^2 - \frac{k(k+1)^2}{4} \right)$$

服从自由度为 $k - 1$ 的 χ^2 分布

其中 N, k 表示数据集和算法数目

若 k 较小 ($k < 30$)， χ^2 倾向于无显著差别，所以通常使用变量

$$\tau_F = \frac{(N-1)\tau_{\chi^2}}{N(k-1) - \tau_{\chi^2}}$$

τ_F 服从自由度为 $k-1$ 和 $(k-1)(N-1)$ 的 F 分布

Nemenyi后续检验

若“所有算法的性能相同”这个假设被拒绝，说明算法的性能显著不同，此时可用Nemenyi后续检验进一步区分算法。

Nemenyi检验计算平均序值差别的临界阈值

$$CD = q_{\alpha} \sqrt{\frac{k(k+1)}{6N}}$$

如果两个算法的平均序值之差超出了临界阈值 CD ，则以相应的置信度拒绝“两个算法性能相同”这一假设。

表 2.7 Nemenyi 检验中常用的 q_{α} 值

α	算法个数 k								
	2	3	4	5	6	7	8	9	10
0.05	1.960	2.344	2.569	2.728	2.850	2.949	3.031	3.102	3.164
0.1	1.645	2.052	2.291	2.459	2.589	2.693	2.780	2.855	2.920

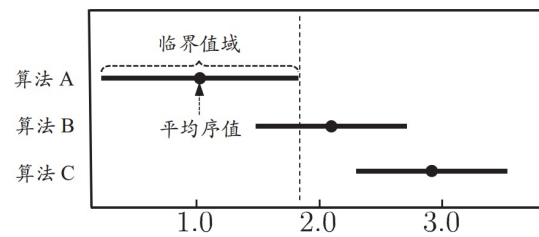
常用的后续检验方法包括Nemenyi test（对比 k 个算法相互之间的性能）和Bonferroni-Dunn test（将某个算法与其余 $k-1$ 个算法对比）。

Friedman检验图

根据上例的序值结果可绘制如下Friedman检验图，横轴为平均序值，每个算法圆点为其平均序值，线段为临界阈值的大小。

若两个算法有交叠(A和B)，则说明没有显著差别；

否则有显著差别(A和C),算法A明显优于算法C.



46

大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

47

偏差与方差

通过实验可以估计学习算法的泛化性能，而“偏差-方差分解 (bias-variance decomposition)”可以用来帮助解释泛化性能。**偏差-方差分解试图对学习算法期望的泛化错误率进行拆解。**

对测试样本 x ，令 y_D 为 x 在数据集中的标记， y 为 x 的真实标记， $f(x; D)$ 为训练集 D 上学得模型 f 在 x 上的预测输出。以回归任务为例：学习算法的期望预期为：

$$\bar{f}(x) = \mathbb{E}_D[f(x; D)]$$

使用样本数目相同的不同训练集产生的方差为

$$\text{var}(x) = \mathbb{E}_D \left[(f(x; D) - \bar{f}(x))^2 \right]$$

噪声为

$$\varepsilon^2 = \mathbb{E}_D \left[(y_D - y)^2 \right]$$

48

偏差与方差

使用样本数目相同的不同训练集产生的方差为

$$\text{var}(x) = \mathbb{E}_D \left[(f(x; D) - \bar{f}(x))^2 \right]$$

期望输出与真实标记的差别称为偏差，即 $\text{bias}^2(x) = (\bar{f}(x) - y)^2$ 为便与讨论，假定噪声期望为0，也即 $\mathbb{E}_D[y_D - y] = 0$ ，对泛化误差分解

$$\begin{aligned} E(f; D) &= \mathbb{E}_D \left[(f(x; D) - y_D)^2 \right] \\ &= \mathbb{E}_D \left[(f(x; D) - \bar{f}(x) + \bar{f}(x) - y_D)^2 \right] \\ &= \mathbb{E}_D \left[(f(x; D) - \bar{f}(x))^2 \right] + \mathbb{E}_D \left[(\bar{f}(x) - y_D)^2 \right] \\ &\quad + \mathbb{E}_D \left[2(f(x; D) - \bar{f}(x))(\bar{f}(x) - y_D) \right] \\ &= \mathbb{E}_D \left[(f(x; D) - \bar{f}(x))^2 \right] + \mathbb{E}_D \left[(\bar{f}(x) - y_D)^2 \right] \end{aligned}$$

期望的运算性质
 $E[AX + B] = AE[X] + B$
 $E[XY] = E[X]E[Y]$

49

偏差与方差

$$\begin{aligned}
 &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y + y - y_D)^2 \right] \\
 &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y)^2 \right] + \mathbb{E}_D \left[(y - y_D)^2 \right] \\
 &\quad + 2\mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y)(y - y_D) \right]
 \end{aligned}$$

又由假设中噪声期望为0，可得

$$E(f; D) = \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + (\bar{f}(\mathbf{x}) - y)^2 + \mathbb{E}_D \left[(y_D - y)^2 \right]$$

于是： $E(f; D) = \text{bias}^2(\mathbf{x}) + \text{var}(\mathbf{x}) + \varepsilon^2$

也即泛化误差可分解为偏差、方差与噪声之和。

50

偏差与方差

- 偏差度量了学习算法期望预测与真实结果的偏离程度；即刻画了学习算法本身的拟合能力；
- 方差度量了同样大小训练集的变动所导致的学习性能的变化；即刻画了数据扰动所造成的影响；
- 噪声表达了在当前任务上任何学习算法所能达到的期望泛化误差的下界；即刻画了学习问题本身的难度。

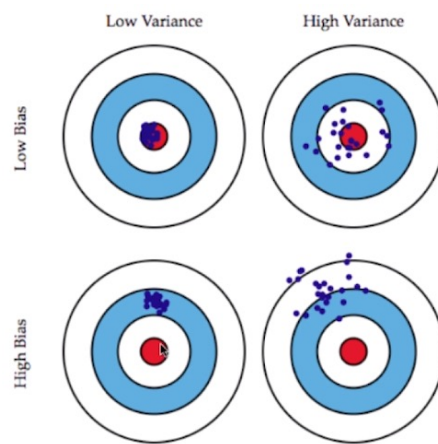
泛化性能是由学习算法的能力、数据的充分性以及学习任务本身的难度所共同决定的。给定学习任务为了取得好的泛化性能，需要使偏差小(充分拟合数据)而且方差较小(减少数据扰动产生的影响)。

51

偏差方差权衡 (Bias Variance Trade off)

偏差:是衡量预测值与真实值的关系,是指预测值与真实值之间的差值.

方差:是衡量预测值之间的关系,和真实值无关.也就是他们的离散程度



如何解决高偏差和高方差?

高偏差:

1. 增加模型复杂度
2. 增加特征
3. 减少正则化程度

高方差:

1. 降低模型复杂度
2. 增加训练数据
3. 减少特征数量
4. 增加正则化程度
5. 使用验证集

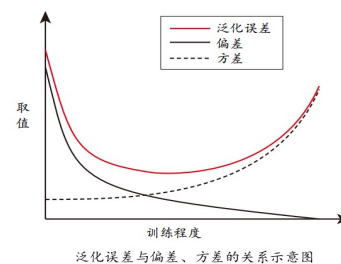
52

偏差与方差

一般来说, 偏差与方差是有冲突的, 称为偏差-方差窘境。

如右图所示, 假如我们能控制算法的训练程度:

- 在训练不足时, 学习器拟合能力不强, 训练数据的扰动不足以使学习器的拟合能力产生显著变化, 此时偏差主导泛化错误率;
- 随着训练程度加深, 学习器拟合能力逐渐增强, 方差逐渐主导泛化错误率;
- 训练充足后, 学习器的拟合能力非常强, 训练数据的轻微扰动都会导致学习器的显著变化, 若训练数据自身非全局特性被学到则会发生过拟合。



泛化误差与偏差、方差的关系示意图

53

大纲

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差
- 阅读材料

54

阅读材料

- 自助采样法在机器学习中有重要用途,[Efron and Tibshirani, 1993]对此有详细讨论。
- ROC曲线在二十世纪八十年代后期被引入机器学习[Spackman, 1989],AUC则是从九十年代中期起在机器学习领域广为使用 [Bradley,1997].[Hand and Till,2001]将ROC曲线从二分类任务推广到多分类任务.[Fawcett,2006]综述了ROC曲线的用途。
- [Drummond and Holte,2006]发明了代价曲线.代价敏感学习 [Elkan,2001;Zhou and Liu,2006]专门研究非均等代价下的学习。

55

阅读材料

- [Dietterich,1998]指出了常规k折交叉验证法存在的风险,并提出了5*2折交叉验证法.[Demsar, 2006]讨论了对多个算法进行比较检验的方法.
- [Geman et al.,1992]针对回归任务给出了偏差-方差-协方差分解,后来被简称为偏差-方差分解。但仅基于均方误差的回归任务中推导,对分类任务,由于0/1损失函数的跳变性,理论上推导出偏差-方差分解很困难。已有多种方法可通过实验对偏差和方差进行估计 [Kong and Dietterich,1995;Kohavi and Wolpert, 1996; Breiman,1996;Friedman,1997;Domingos,2000].